LEVEL 4 OF 8 · THE DBT FACTORY

Everyone has their own "active user."

Two levels in, and you've fixed two specific queries. But this morning Diego pulled up a dashboard, Priya pulled up a different one, and Farrah pulled up a spreadsheet — all three claim to show "active users" for the same week, all three disagree. You go looking for the source query behind each one. There are five. Buried in an old repo, you also find a folder called **dbt/** — Jordan started building something to end this, and never finished it.

UNLOCKS: DBT ACCESS

1 — 2 — 3 — 4 — 5 — 6 — 7 — 8

CASE FILE

Five queries, five definitions of "active"

You pull the source query behind every dashboard claiming to show "active users." None of them match.

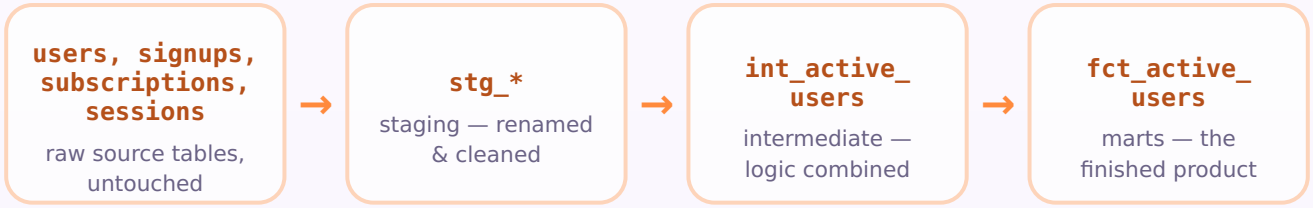
Owner	Where it lives	Their definition of "active"
Priya (Growth)	Acquisition dashboard	Logged in at least once this week
Diego (Product)	Feature-usage dashboard	Used a core feature in the last 14 days
Farrah (Finance)	Revenue spreadsheet	Has a subscription in *active* status, regardless of usage
Support	Ticket system export	Opened the app in the last 30 days
Jordan (unfinished)	`dbt/staging/stg_users.sql`	Session in the last 7 days *and* subscription active — never shipped

Five queries. Five numbers. Zero agreement — and every team is right, by their own definition.

WHITEBOARD MOMENT

staging → intermediate → marts

Jordan's unfinished folder was the right idea: one factory line, raw tables in on the left, one trustworthy number out on the right.



Nobody queries staging or intermediate models directly — they're work-in-progress stations on the line. The mart is the only finished product that leaves the factory floor, and it's the only thing Priya, Diego, and Farrah should ever be pointed at.

BUILD IT YOURSELF

Your first dbt model

Start the factory line. Fill in the three blanks to finish this staging model.

```
-- models/staging/stg_subscriptions.sql
select
    id as subscription_id,
    user_id,
    status as subscription_status,
    ___________ as started_at,
    ___________ as ended_at
from ___________('solace', 'subscriptions')
```

Blank 1 — which raw column tracks when the subscription actually began?

Blank 2 — which raw column tracks when it ended, if it did?

Blank 3 — which Jinja function wraps a raw table name, since this is a source table dbt didn't build?

SQL ESCAPE ROOM · LOCKED

The model that wouldn't compile

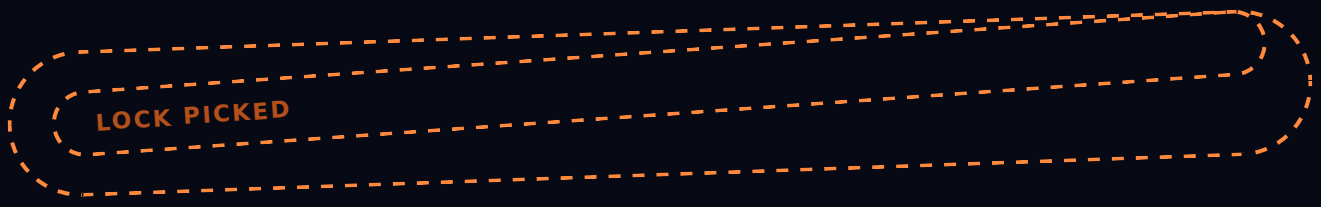
LOCKED — you can't move to the next mission until you find the bug.

Your intermediate model won't build. dbt throws a compile error before it even reaches the database:

```
-- models/intermediate/int_active_users.sql
select
    u.user_id,
    s.subscription_status,
    sess.last_session_at
from {{ ref('stg_users') }} u
join ref('stg_subscriptions') s on u.user_id = s.user_id
join {{ ref('stg_sessions') }} sess on u.user_id = sess.user_id
where s.subscription_status = 'active'
```

What's wrong with this model? Write it before you turn the page.

No peeking — the fix is explained on the next page, not this one.



The missing curly braces

`ref()` is a Jinja function — it only gets resolved into a real table reference when it's wrapped in `{{ }}`. On the subscriptions line, the braces are missing, so dbt treats `ref('stg_subscriptions')` as literal SQL text instead of a template call. The database has no idea what a function called `ref` is, so the model fails to compile before it ever runs. It's one of the most common first-week dbt mistakes: writing something that looks like valid SQL but isn't valid Jinja.

CHOOSE YOUR STRATEGY

Ephemeral vs. view vs. table

Every dbt model needs a materialization. No single right answer — each trades freshness for cost differently.

Materialization	How it works	Trade-off
1. Ephemeral	Inlined directly into whatever model references it — no database object gets built at all.	Zero storage cost, but you can't query it on its own to debug — it only exists inside the SQL of whatever calls it.
2. View	A saved query that recomputes from scratch every time something selects from it.	Always reflects the latest data with no storage cost, but slow if the underlying logic is heavy and queried often.
3. Table	Fully materialized on a schedule — the data is physically written and sits there until the next run.	Fast to query no matter how heavy the logic is, but uses storage and is only as fresh as the last scheduled run.

Solace's canonical active-user model ships as a table this level — it's queried by every dashboard on the floor, so scheduled freshness beats recomputing the same logic on every page load.

DETECTIVE CASE

Why did the model break in production?

CONFIDENTIAL

Monday morning: **fct_active_users** shows zero rows for the past three days. Priya's dashboard is empty. Diego's is empty. Nobody touched the model since it shipped Friday — the run just started failing on its own, silently, over the weekend.

*Root cause: Solace's billing provider added a new subscription status, '**trialing_extended**', on Saturday. The model's CASE statement only knew about active, cancelled, and trialing — the new value fell through to NULL, tripped a not-null test, and the whole run stopped rather than ship bad data.*

REFLECTION CARDS

Before you move on

Where in your own work does a hardcoded list of values (a CASE statement, an IN list) quietly assume nothing new will ever show up?

Why is a model that fails loudly on a bad value better than one that ships a silently wrong number?

What would you have needed to notice this Saturday instead of Monday?

SENIOR ENGINEER NOTES

From someone who's seen this before

A model that fails a test is doing its job. The failure you should actually worry about is the one nobody wrote a test for.

Handy one nobody teaches you up front: `dbt run --select +fct_active_users` rebuilds a model and everything it depends on, without rebuilding the entire project — the `+` is the whole trick.

Every "we didn't expect that value" bug I've shipped had the same root cause: a CASE statement written for the values that existed on the day I wrote it, not the values that would eventually exist.

BOSS FIGHT

Ending the "Whose Number Is Right" War

The canonical model is built and tested. Now Priya, Diego, and Farrah all need to move their dashboards onto it — which means all three have to admit, in some form, that their old number was off.

- [ ] **A — Quietly point every dashboard at the new table, no explanation.**
  Trade-off: fast, but the numbers just shift overnight with no warning — looks suspicious the moment anyone compares to last week.

- [ ] **B — Walk all three through the model's logic together, take questions, then switch the dashboards.**
  Trade-off: an uncomfortable meeting where everyone has to reconcile with their old number, but the model lands with real buy-in instead of being imposed.

- [ ] **C — Deprecate the old queries immediately and force the switch without a meeting.**
  Trade-off: fastest technically, but breeds resentment — nobody understands why their number changed or trusts the new one yet.

B. Same principle every level so far: the version of you that gets trusted with bigger problems is the one who brings people into the fix, not the one who just ships it at them.

UNLOCK

dbt Cheat Cards

staging → intermediate → marts

Staging: 1:1 with a source, cleaned and renamed. Intermediate: business logic combined across staging models. Marts: the finished, wide table dashboards actually query.

ref() vs. source()

source() points to a raw table dbt didn't build. ref() points to another dbt model — dbt uses every ref() to build its dependency graph and run models in the right order automatically.

Materializations

View: always fresh, no storage. Table: materialized, fast to query, needs scheduling. Ephemeral: inlined into whatever references it — no database object at all.

Tests

unique and not_null catch most bugs before a dashboard ever sees them. relationships tests catch orphaned foreign keys. Run tests before you trust a model, not after someone complains.

Keep this one next to the SQL cheat cards from Level 2 and 3 — this is where the pipeline starts owning the checks, not you.

CHEATSHEET

Level 4, condensed

The problem: five queries across Growth, Product, Finance, Support, and Jordan's abandoned model all defined "active user" differently.

The fix: one canonical dbt model — staging → intermediate → marts — that every dashboard now points at.

The compile bug: missing {{ }} around ref() breaks the Jinja templating, not the SQL underneath it.

The Boss Fight call: bring every stakeholder into the model's logic before switching their dashboards, not after.

FINAL PROJECT · PART 1

Test Yourself on Solace's dbt Project

Three questions, one real answer each. No skipping ahead, no peeking at the next page early. Pick your answers below, then turn the page to see how you did.

1. What does {{ ref('stg_subscriptions') }} actually do inside a dbt model?

- ○ A. Nothing — it's just a comment
- ○ B. Tells dbt this model depends on stg_subscriptions, so it's built first and resolved to the right table name
- ○ C. Copies the entire stg_subscriptions table into this model's output

2. Why did Solace end up with five different "active user" numbers?

- ○ A. Bad SQL syntax in all five queries
- ○ B. Each team built their own definition off a different table, with nobody owning one canonical version
- ○ C. The database was corrupted

3. Why materialize the canonical active-user model as a table instead of a view?

- ○ A. Tables are always faster to write
- ○ B. It's queried by many dashboards, so a scheduled table avoids recomputing the same logic on every page load
- ○ C. Views don't support JOINs

Your verdict — and the answer key — is one page away.

CASE FILE VERIFIED

How'd you do?

1 → B. ref() is what lets dbt build its dependency graph and resolve the correct table name at compile time.

2 → B. Five teams, five source tables, zero canonical owner — the definitions weren't wrong, they were just never reconciled.

3 → B. A table pays the storage cost once per scheduled run instead of every dashboard load recomputing the same logic.

Got all 3?

You're already thinking like the person who ends a "whose number is right" war for good instead of refereeing it every quarter.

Got 2 out of 3?

Solid — you're tracing the dependency graph, not just the SQL. Flip back to Page 6 and Page 3 for the one that slipped past.

Got 1 or fewer?

No penalty for that in Level 4: The dbt Factory — dbt's mental model takes a minute to click. Re-read the Cheatsheet on Page 13 and carry it into Level 5.

FINAL PROJECT · PART 2

The Canonical Active-User Model

No answer key here — map out a metric from your own work that has more than one competing definition, the way "active user" did at Solace.

Your Case

Metric with competing definitions	Where each definition currently lives	What the canonical definition should be	Who needs to sign off before it ships
CLUE #1 — Which metric has more than one definition floating around your org?			
CLUE #2 — What should the one canonical definition actually be, and why?			
CLUE #3 — Whose buy-in do you need before you can retire the other versions?			

A fourth real artifact for your Level 8 portfolio project — this one shows you can end an argument permanently, not just win it once.

LEVEL COMPLETE

World 4: The dbt Factory

You finished Level 4 and ended a five-way argument with one model everyone actually trusts.

Priya, Diego, and Farrah now all pull from the same table without checking with each other first — the first time that's happened since you started.

"*A pipeline you build once and trust beats a query you rerun every time you don't.*"

NEXT UP

Level 5: The Warehouse City

You've built one clean model. Next, you design where every model in Solace's warehouse actually lives.